Software Requirements Specification (SRS)

Zomato Clone - Food Ordering & Delivery Application

Document Version:	1.0.0	**Standard:**	IEEE Std 830-1998 Format
Developer / Intern:	Mohit Sharma	**Senior Mentor:**	Mentor Aryaman
Internship Program:	Web Development Internship	**Date:**	July 24, 2026
Primary Tech Stack:	MERN Stack (MongoDB, Express, React, Node)	**State & Styling:**	Redux Toolkit, Tailwind CSS

Executive Summary & Project Overview

This Software Requirements Specification (SRS) document provides a formal, comprehensive architectural specification for the **Zomato Clone** application. Developed by **Mohit Sharma** under the mentorship of **Mentor Aryaman** as part of the **Web Development Internship Program**, this project replicates the end-to-end user workflows of commercial food delivery platforms. Built on the modern MERN stack architecture, the application integrates RESTful API communication, stateless authentication, real-time client-side proximity distance calculations, promotional menu indexing, and Redux Toolkit state management.

1. Introduction

1.1 Purpose

The purpose of this SRS document is to define the complete functional and non-functional requirements governing the implementation and deployment of the Zomato Clone system. It serves as the primary technical baseline for academic and internship performance evaluation, establishing clear benchmarks for software quality, system security, modular backend API architecture, and responsive frontend interaction models.

Furthermore, this document outlines explicit operational boundaries, component interactions, database schemas, data flow models, and client-server communication standards. It is intended for software architects, backend engineers, frontend developers, and evaluators reviewing the codebase.

1.2 Scope & System Boundaries

The **Zomato Clone** software encompasses a full-fledged e-commerce food ordering platform operating across decoupled client and server environments. Key functional boundaries include:

- **User Authentication & Authorization:** User registration, password encryption via bcrypt, login authentication, JWT session token emission, and route protection middleware.
- **Restaurant Discovery Engine:** Multi-criteria filtering (cuisine, min rating 4.0+, max delivery duration <30m), text search, and real-time proximity sorting via GPS geolocation.
- **Interactive Menu Inspection:** Category-based dish listings, Veg/Non-Veg indicators, and promotional badge tagging (■ *Bestseller*, ★ *Top Rated*, ■ *Chef's Special*).
- **Cart Management & Pricing Rules:** Quantity modification, single-restaurant cart enforcement, automatic tax calculation (5% GST), and delivery fee logic.
- **Checkout & Order Tracking:** Order placement, payment method selection, order confirmation, and historical order status tracking.

1.3 Definitions, Acronyms, and Abbreviations

Acronym / Term	Full Definition & Technical Meaning
SRS	Software Requirements Specification formatted according to IEEE Std 830-1998 standards.
MERN Stack	MongoDB (Database), Express.js (Backend Framework), React.js (Frontend SPA), Node.js (Runtime).
JWT	JSON Web Token - URL-safe compact token format used for stateless user authorization headers.
REST API	Representational State Transfer - Architectural style for HTTP services exchanging JSON payloads.
Redux Toolkit	Official standard toolset for managing global application state in React applications.
Mongoose ODM	Object Document Mapper providing schema validation and query building for MongoDB.
SPA	Single Page Application - Web app loading a single HTML page and dynamically updating content.
Haversine Formula	Mathematical equation calculating great-circle distances between two geographic coordinates.
Bcrypt	Adaptive password-hashing function based on the Blowfish cipher to prevent brute-force attacks.
CORS	Cross-Origin Resource Sharing protocol allowing controlled browser access across distinct domains.

1.4 System Overview & Architecture

The system adopts a modern multi-tier client-server architectural model. The user client consists of a React 18 single-page application bundled with Vite, styled using Tailwind CSS utility classes, and driven by a Redux Toolkit centralized store. The client application executes on local port 5173.

The backend layer comprises a Node.js runtime hosting an Express application listening on local port 5000 under base API path http://localhost:5000/api/v1. The backend enforces security policies, handles JWT validation middleware, executes business logic, and manages data queries against a local MongoDB database instance operating on port 27017.

2. Overall Description

2.1 Product Perspective

The Zomato Clone is an independent, autonomous software application designed for desktop and mobile web environments. The application operates as a standalone decoupled deployment: the frontend UI handles client presentation and view state without directly invoking database drivers, communicating strictly via HTTP REST JSON requests.

For location services, the system interfaces with OpenStreetMap's Nominatim REST API to perform free forward and reverse geocoding without relying on external proprietary vendor keys.

2.2 User Classes & Characteristics

The software accommodates four primary user classes:

1. **Customer (End User):** Primary consumer persona. Requires an intuitive interface to search food items, discover nearby restaurants, configure cart items, check out, and track active delivery orders.
2. **Demo Evaluator / Assessor:** Evaluator inspecting project capabilities. Utilizes pre-seeded test credentials (demo@example.com / password123) to quickly test protected routes without manual signup.
3. **Restaurant Partner:** Business entity owning restaurant profiles and managing food item menus.
4. **System Administrator:** Maintains backend server health, monitors database performance, and triggers automated seeding scripts.

2.3 Operating Environment

The Zomato Clone system operates across distributed web browser runtimes and server hosting platforms. The detailed specification of the operational environment is categorized as follows:

Environment Tier	Hardware & Software Specifications
Client Web Browser	HTML5 compliant web browser (Google Chrome v100+, Mozilla Firefox v100+, MS Edge v100+, Apple Safari v15+). Minimum 2 GB RAM, dual-core CPU, and 1024x768 screen resolution.
Server Runtime	Node.js runtime environment (v18.x or v20.x LTS). Host server requiring 1 CPU core, 1 GB RAM minimum, listening on TCP port 5000.
Database Server	MongoDB Community Server (v6.0+ or v7.0+) running locally on 127.0.0.1:27017. Stores documents in BSON format with Mongoose schema validation.
External APIs	OpenStreetMap Nominatim Reverse & Forward Geocoding REST API over HTTPS protocol.

2.4 Design and Implementation Constraints

Several architectural and technical constraints govern the software implementation:

- **Single-Restaurant Cart Enforcement Rule:** The cart state enforces exclusive ordering from a single kitchen. If a user selects an item from a different restaurant, the system automatically prompts and clears existing cart contents before inserting the new dish.
- **Stateless Token Security:** User session authorization relies exclusively on signed JSON Web Tokens (JWT) transmitted via Authorization: Bearer headers. No session state is held in server memory.
- **Offline Sandbox Seeding:** To ensure predictable testing without requiring external administrative dashboards during evaluation, the repository includes a programmatic seeder (`seedData.js`) that populates 50 realistic Indian restaurants and 2,500 food items into MongoDB.
- **Geospatial Computation Model:** Distance calculations are calculated dynamically on the client using the Haversine equation, avoiding expensive database geo-index setup while maintaining offline local development compatibility.
- **Vite Bundling Limits:** Frontend distribution chunks must compile under 350 kB gzip size to ensure sub-2 second render performance.

2.5 Assumptions and Dependencies

The software architecture operates under the following explicit assumptions:

1. The local execution machine has Node.js and MongoDB installed and running without port conflicts on 5000 and 27017.
2. Client web browsers support HTML5 Geolocation APIs and JavaScript ES6+ features.
3. Unsplash CDN images used for restaurant covers and food items remain accessible over network interfaces.

3. Specific Functional Requirements

3.1 Functional Requirement Set 1: User Authentication & Authorization

FR-1.1 User Registration (Signup):
• *Inputs:* Full Name, Email Address, Password.
• *Processing:* Validates email syntax; checks for duplicate accounts; hashes plain text password using `bcrypt` (10 salt rounds); creates `User` document; generates signed JWT token (expires in 7 days).
• *Outputs:* HTTP 201 Created response containing user profile and token; updates Redux `auth` state slice.

FR-1.2 User Authentication (Login):
• *Inputs:* User Email, Password (supports quick click access for `demo@example.com`).
• *Processing:* Queries `User` document including hidden password field; verifies password against bcrypt hash; issues fresh JWT session token.
• *Outputs:* HTTP 200 OK response; persists token in browser `localStorage`; grants access to user privileges.

FR-1.3 Authorization & Protected Routes:
• *Processing:* Client `ProtectedRoute.jsx` intercepts route changes to `/checkout` and `/orders`. Backend `authMiddleware.js` extracts Bearer token from headers and attaches `req.user`.
• *Outputs:* Unauthenticated users attempting to access checkout are redirected to `/login` with toast notification.

3.2 Functional Requirement Set 2: Restaurant & Menu Browsing Engine

FR-2.1 Restaurant Catalog & Dynamic Filtering:
• *Processing:* Fetches dataset via `GET /api/v1/restaurants`. Filters client-side by search text query, selected cuisine pill (Italian, American, Indian, Fast Food, Desserts), star rating (4.0+ toggles), and maximum delivery time (<30 mins).
• *Outputs:* Updated restaurant grid view rendering matched restaurant cards.

FR-2.2 Interactive Location & Proximity Engine:
• *Inputs:* HTML5 GPS coordinates or manual location input string (e.g. 'Mumbai', 'Indiranagar').
• *Processing:* Queries OpenStreetMap Nominatim API for geocoding coordinates. Computes physical distance in kilometers using the Haversine formula:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2(\Delta lat/2) + \cos(lat1)\cdot\cos(lat2)\cdot\sin^2(\Delta lon/2)}\right)$$

Sorts restaurants ascending by distance (closest first) and renders floating distance badges (`2.4 km away`).
• *Outputs:* Location-aware restaurant catalog sorted by proximity.

FR-2.3 Menu Item Display & Promotional Badges:
• *Processing:* Queries menu items via `GET /api/v1/food-items?restaurant=:id`. Groups dishes by category. Displays Veg/Non-Veg indicators and promotional badges (`■ Bestseller`, `★ Top Rated`, `■ Chef's Special`).
• *Outputs:* Menu page rendering categorized dishes with item cards.

3.3 Functional Requirement Set 3: Cart Management & Pricing Engine

FR-3.1 Cart State & Quantity Control:
• *Processing:* Dispatches `addToCart` and `removeFromCart` Redux actions. Manages item quantities per dish.
• *Enforcement:* If an item from a different restaurant is selected, clears existing cart automatically.
• *Outputs:* Updated Redux cart state, header cart count badge, and floating cart footer.

FR-3.2 Dynamic Pricing Breakdown:
• *Calculations:* Dynamically computes:
  • *Item Subtotal* = Σ(item.price × item.quantity)
  • *Delivery Fee* = Flat ■45 (waived for subtotals exceeding ■500)
  • *Taxes & Charges* = 5% GST on Subtotal
  • *Grand Total* = Item Subtotal + Delivery Fee + Taxes
• *Outputs:* Itemized receipt summary rendered on Cart & Checkout views.

3.4 Functional Requirement Set 4: Checkout & Order Processing

FR-4.1 Order Placement & Validation:
• *Inputs:* Selected delivery address, payment option ('Cash on Delivery', 'UPI / Card').
• *Processing:* Submits `POST /api/v1/orders` containing restaurant ID, item array, total amount, and delivery address. Clears Redux cart upon HTTP 201 response.
• *Outputs:* HTTP 201 Created response containing newly created Order document.

FR-4.2 Order Confirmation & Tracking:
• *Processing:* Navigates user to `/order-success/:orderId`. Provides navigation to `/orders` displaying historical order list with status badges (`Placed`, `Preparing`, `Out for Delivery`, `Delivered`).
• *Outputs:* Order confirmation page and interactive order history screen.

4. External Interface Requirements

4.1 User Interface (UI) Design Standards

The UI design follows modern visual standards using Tailwind CSS and custom typography:

- **Theme Palette:** Zomato Pure Crimson (`#E23744`), Dark Slate (`#0F172A`), Emerald Green (`#059669`), and Slate Grey (`#64748B`).
- **Navigation Bar:** Top fixed bar with brand logo, location selector, global search, cart counter, and user profile state.
- **Hero Section:** Dark overlay section featuring radial lighting, search inputs, and location selection panel.
- **Image Animations:** Image cards incorporate loading skeletons, error fallback handlers, and blur-to-focus transition effects.

4.2 Hardware & Software Interfaces

- **REST API Endpoints:** Base path `http://localhost:5000/api/v1` for `/auth`, `/restaurants`, `/food-items`, and `/orders`.
- **OpenStreetMap Nominatim:** External HTTPS REST service for forward and reverse address lookup.
- **CORS Middleware:** Express server configured with `cors()` middleware allowing requests from client origin `http://localhost:5173`.

5. Non-Functional Requirements (NFRs)

NFR Category	Detailed Technical Requirements & Metrics
5.1 Performance	• Frontend build assets compiled via Vite under 325 kB total bundle size. • Initial DOM paint under 1.5 seconds on standard 4G network connections. • Database queries optimized with single-field indexes (`_id`, `email`, `restaurant`), delivering response times under 50ms.
5.2 Security	• Passwords encrypted using bcrypt algorithm with a salt factor of 10. • Authentication routes protected via stateless JWT validation middleware. • MongoDB ObjectIDs sanitized using `validateObjectId.js` middleware to prevent injection attacks.
5.3 Reliability	• Health check endpoint (`GET /api/v1/health`) returning server status. • Centralized error handling middleware (`errorMiddleware.js`) preventing process crashes and returning structured JSON error tracebacks.
5.4 Maintainability	• Backend structured according to Model-View-Controller (MVC) architectural pattern. • Frontend store modularized into clean Redux slices (`authSlice`, `cartSlice`, `restaurantSlice`).

6. Database & Data Requirements

The system utilizes MongoDB as its primary database engine. Mongoose ORM enforces schema validation across four collections:

6.1 User Model Schema (`backend/src/models/User.js`)

```
{ name: { type: String, required: [true, 'Name is required'], trim: true }, email: { type: String, required: [true, 'Email is required'], unique: true, lowercase: true }, password: { type: String, required: [true, 'Password is required'], select: false }, role: { type: String, enum: ['customer', 'partner', 'admin'], default: 'customer' }, addresses: [{ type: String }], timestamps: true }
```

6.2 Restaurant Model Schema (`backend/src/models/Restaurant.js`)

```
{ name: { type: String, required: [true, 'Restaurant name is required'] }, image: { type: String, required: [true, 'Restaurant image URL is required'] }, cuisine: [{ type: String }], rating: { type: Number, default: 4.0, min: 0, max: 5 }, deliveryTime: { type: Number, required: [true, 'Delivery time is required'] }, costForTwo: { type: Number, required: [true, 'Cost for two is required'] }, address: { type: String, required: [true, 'Address is required'] }, coordinates: { lat: { type: Number }, lon: { type: Number } }, isFeatured: { type: Boolean, default: false }, partner: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }, timestamps: true }
```

6.3 FoodItem Model Schema (`backend/src/models/FoodItem.js`)

```
{ name: { type: String, required: [true, 'Food item name is required'] }, description: { type: String, required: [true, 'Description is required'] }, price: { type: Number, required: [true, 'Price is required'], min: 0 }, category: { type: String, required: [true, 'Category is required'] }, isVeg: { type: Boolean, default: true }, isBestselling: { type: Boolean, default: false }, isTopRated: { type: Boolean, default: false }, isTodaysSpecial: { type: Boolean, default: false }, image: { type: String, required: [true, 'Image URL is required'] }, restaurant: { type: mongoose.Schema.Types.ObjectId, ref: 'Restaurant', required: true }, timestamps: true }
```

6.4 Order Model Schema (`backend/src/models/Order.js`)

```
{ user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, restaurant: { type: mongoose.Schema.Types.ObjectId, ref: 'Restaurant', required: true }, items: [{ foodItem: { type: mongoose.Schema.Types.ObjectId, ref: 'FoodItem', required: true }, name: { type: String, required: true }, price: { type: Number, required: true }, quantity: { type: Number, required: true, min: 1 } }], totalAmount: { type: Number, required: true }, deliveryAddress: { type: String, required: true }, paymentMethod: { type: String, enum: ['COD', 'Card', 'UPI'], default: 'COD' }, status: { type: String, enum: ['Placed', 'Preparing', 'Out for Delivery', 'Delivered', 'Cancelled'], default: 'Placed' }, timestamps: true }
```

7. System Verification & Test Plan Matrix

Test Suite ID	Target Feature	Verification Method	Expected Pass Result
TS-01	User Registration	POST /api/v1/auth/signup	HTTP 201 Created & valid JWT emitted.
TS-02	Demo Login	POST /api/v1/auth/login	HTTP 200 OK & demo state initialized.
TS-03	Geocoding Engine	OpenStreetMap API Query	Coordinates resolved & Haversine distance calculated.
TS-04	Single Kitchen Guard	Redux Cart Modification	Previous restaurant items cleared automatically.
TS-05	Order Submission	POST /api/v1/orders	Order created & redirect to Order Success page.

8. Document Approval & Sign-Off

Role / Title	Name	Status / Signature	Date
Developer / Intern	Mohit Sharma	Submitted for Evaluation	July 24, 2026
Senior Mentor	Mentor Aryaman		July 24, 2026